

ANALISIS Y DISEÑO DE ALGORITMOS

Sesión 13:

Algoritmos de Búsqueda Backtracking



Universidad
Tecnológica
del Perú

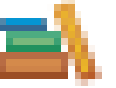
Algoritmos de Búsqueda Backtracking

Ejemplo1: Resolver el Problema de las N Reinas

Descripción:

- El usuario ingresa el tamaño del tablero (N).
- El programa resuelve el problema de las N reinas.
- Muestra la solución visualmente en un tablero

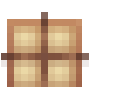
▼  Backtracking

 >  JRE System Library [JavaSE-17]

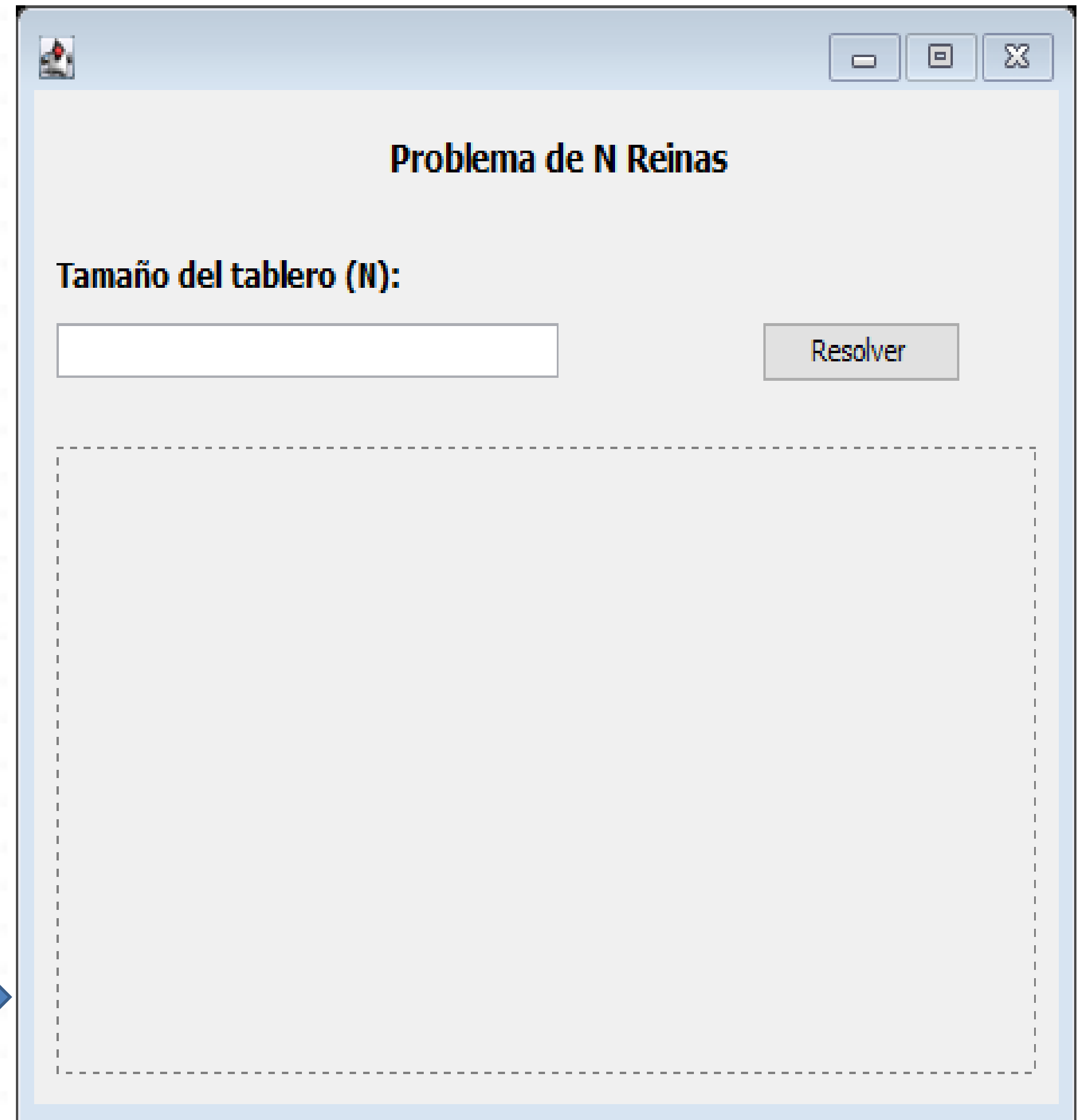
 ▼  src

 ▼  Busquedas

 >  NReinasSolver.java

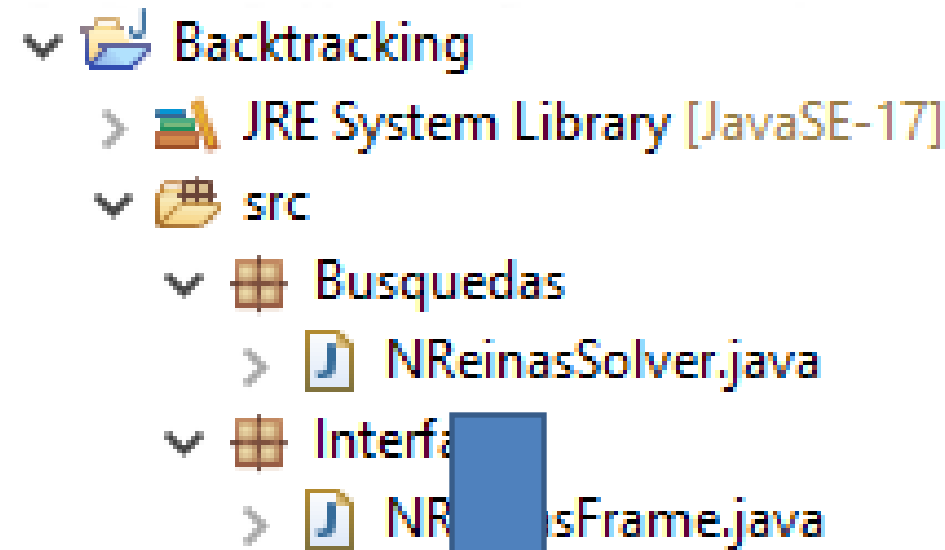
 ▼  Interfaces

 >  NReinasFrame.java



Algoritmos de Búsqueda Backtracking

Ejemplo1: Resolver el Problema de las N Reinas



```
1 package Busquedas;
2
3 public class NReinasSolver {
4
5     private int N;
6     private int[][] tablero;
7
8     public NReinasSolver(int N) {
9         this.N = N;
10        this.tablero = new int[N][N];
11    }
12
13    public boolean resolver() {
14        return resolverReinas(0);
15    }
16 }
```

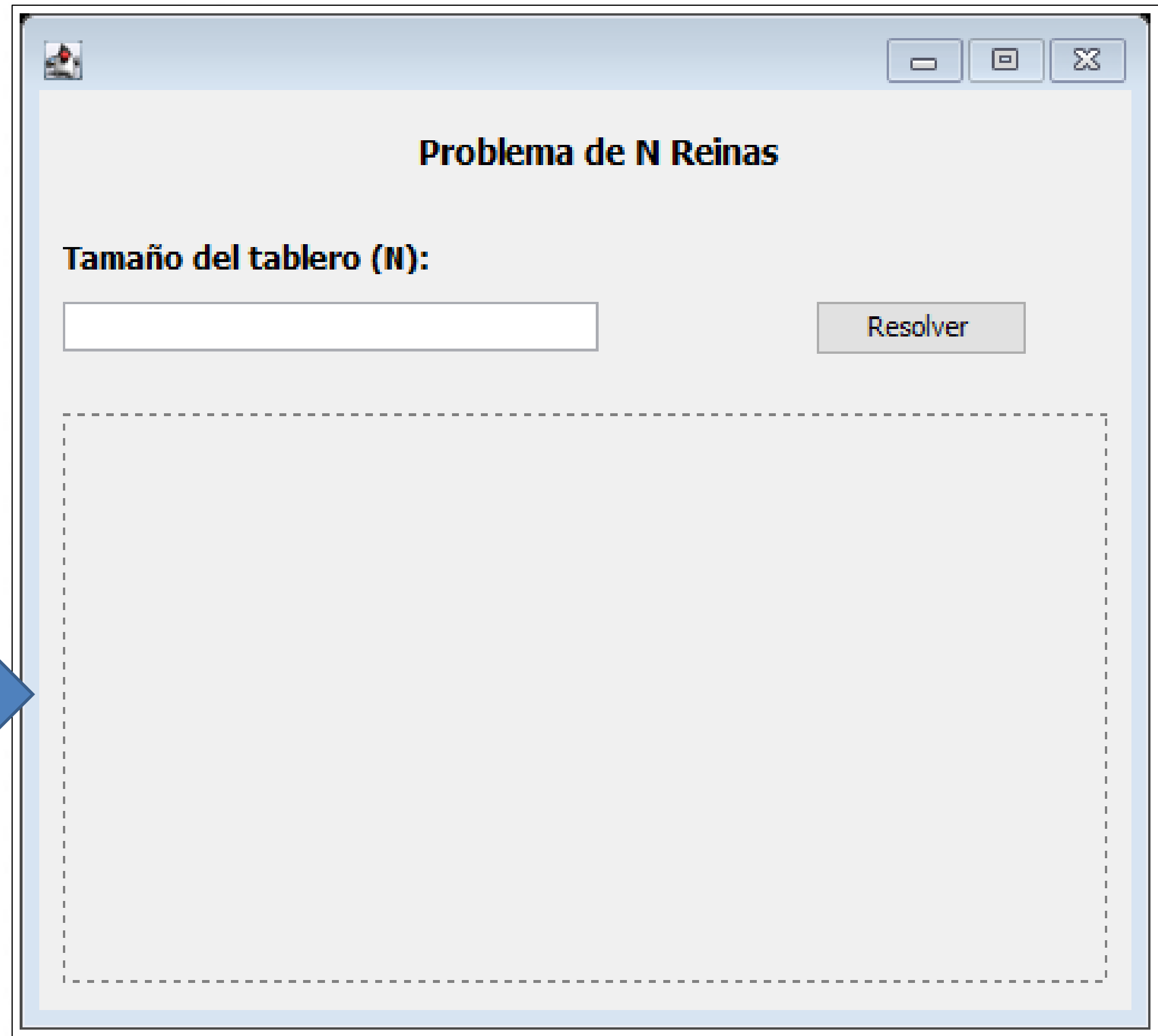
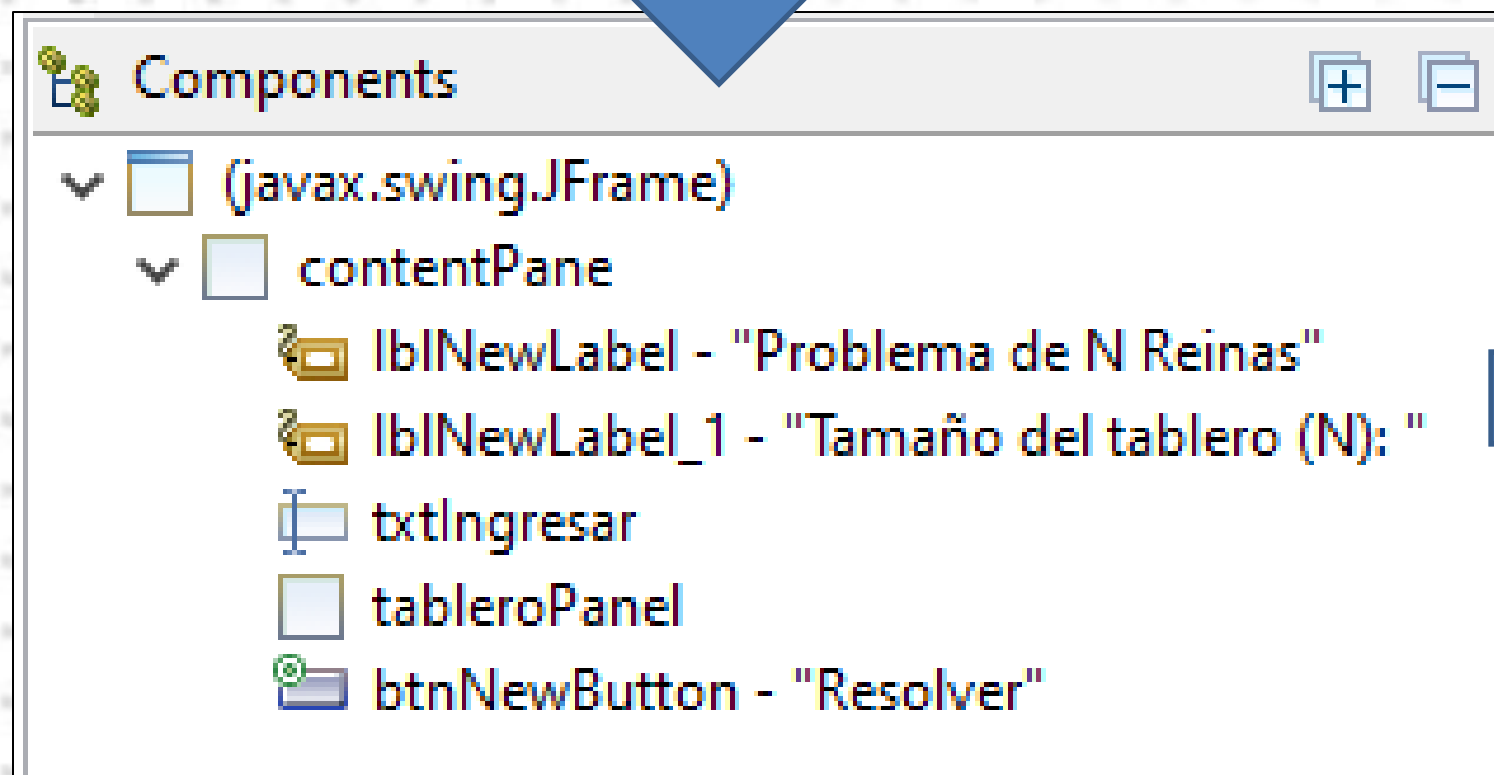
```
17 private boolean resolverReinas(int fila) {
18     if (fila == N) return true;
19
20     for (int col = 0; col < N; col++) {
21         if (esSeguro(fila, col)) {
22             tablero[fila][col] = 1;
23             if (resolverReinas(fila + 1)) return true;
24             tablero[fila][col] = 0;
25         }
26     }
27     return false;
28 }
```

```
30 private boolean esSeguro(int fila, int col) {
31     for (int i = 0; i < fila; i++)
32         if (tablero[i][col] == 1) return false;
33
34     for (int i = fila - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
35         if (tablero[i][j] == 1) return false;
36
37     for (int i = fila - 1, j = col + 1; i >= 0 && j < N; i--, j++)
38         if (tablero[i][j] == 1) return false;
39
40     return true;
41 }
42
43 public int[][] getTablero() {
44     return tablero;
45 }
46 }
```

Algoritmos de Búsqueda Backtracking

Ejemplo1: Resolver el Problema de las N Reinas

- Backtracking
 - JRE System Library [JavaSE-17]
 - src
 - Busquedas
 - NReinasSolver.java
 - Interfaces
 - NReinasFrame.java



Algoritmos de Búsqueda Backtracking

Ejemplo1: Resolver el Problema de las N Reinas

▼ Backtracking
 > JRE System Library [JavaSE-17]
 ▼ src
 ▼ Busquedas
 > NReinasSolver.java
 ▼ Interfaces
 > NReinasFrame.java

```
1 package Interfaces;
2
3 import Busquedas.NReinasSolver;
4
5 import java.awt.Color;
6 import java.awt.EventQueue;
7
8 import javax.swing.JFrame;
9 import javax.swing.JPanel;
10 import javax.swing.border.EmptyBorder;
11 import javax.swing.JLabel;
12 import javax.swing.JOptionPane;
```

```
14 import java.awt.Font;
15 import java.awt.GridLayout;
16
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.BorderFactory;
20 import javax.swing.JButton;
21 import java.awt.event.ActionListener;
22 import java.awt.event.ActionEvent;
23
24 public class NReinasFrame extends JFrame {
25
26     private static final long serialVersionUID = 1L;
27     private JPanel contentPane;
28     private JTextField txtIngresar;
29 }
```

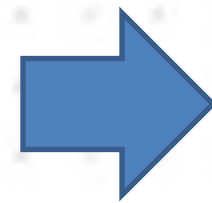

Algoritmos de Búsqueda Backtracking

Ejemplo1: Resolver el Problema de las N Reinas

Símbolos para colocar Reinas:



Resolver



```
 JButton btnNewButton = new JButton("Resolver");
 btnNewButton.addActionListener(new ActionListener() {
     public void actionPerformed(ActionEvent e) {

         int n = Integer.parseInt(txtIngresar.getText());
         NReinasSolver solver = new NReinasSolver(n);
         if (solver.resolver()) {
             mostrarTablero(solver.getTablero());
         } else {
             JOptionPane.showMessageDialog(null, "No se encontró solución.");
         }
     }
 }
```

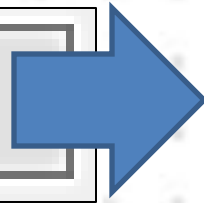
Algoritmos de Búsqueda Backtracking

Ejemplo1: Resolver el Problema de las N Reinas

Símbolos para colocar Reinas:



Resolver

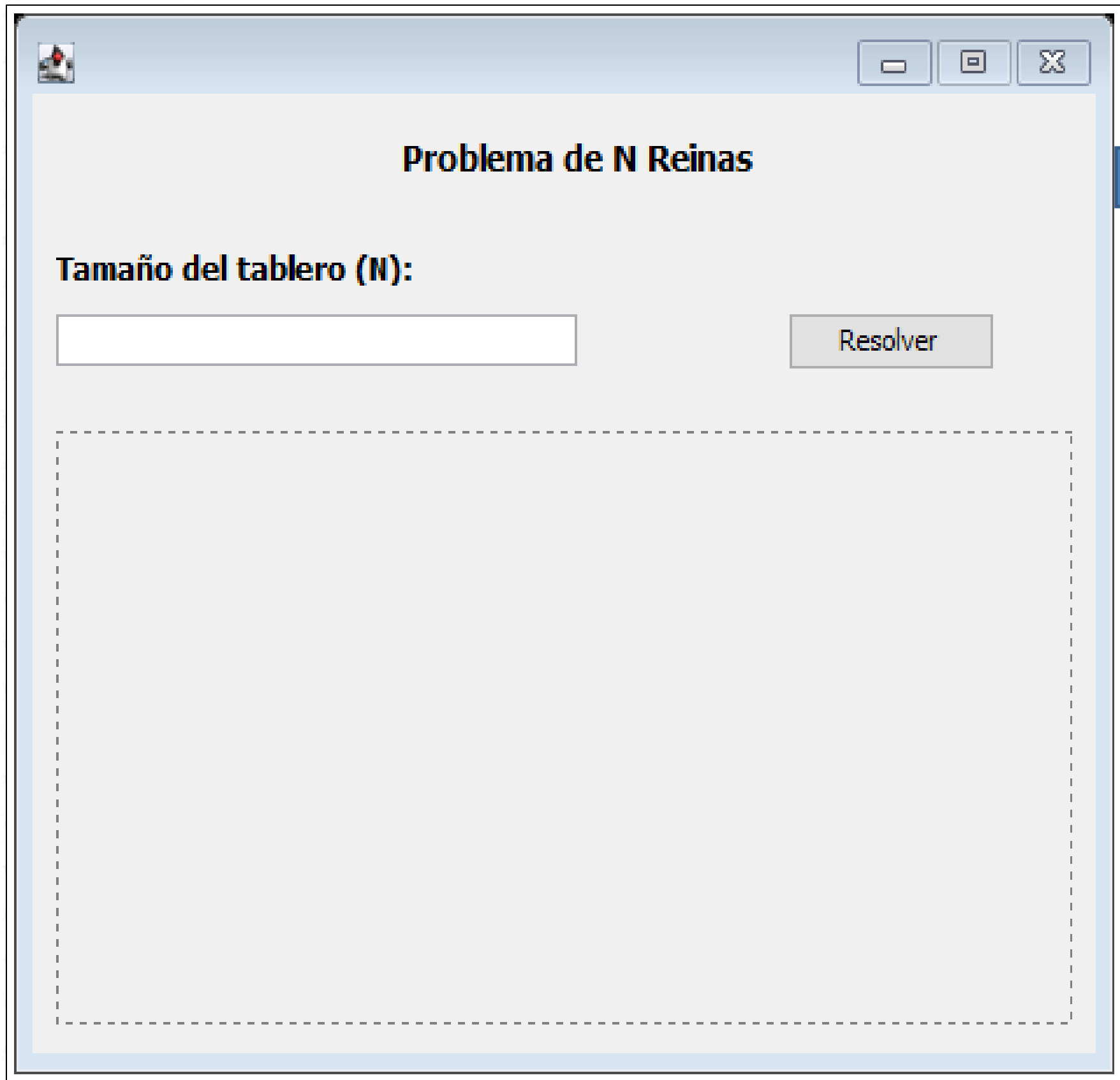


```
private void mostrarTablero(int[][] tablero) {
    tableroPanel.removeAll();
    int N = tablero.length;
    tableroPanel.setLayout(new GridLayout(N, N));
    for (int[] fila : tablero) {
        for (int celda : fila) {
            JLabel label = new JLabel(celda == 1 ? "👑" : "", SwingConstants.CENTER);
            label.setFont(new Font("SansSerif", Font.BOLD, 24));
            label.setBorder(BorderFactory.createLineBorder(Color.BLACK));
            tableroPanel.add(label);
        }
    }
    tableroPanel.revalidate();
    tableroPanel.repaint();
}

});
btnNewButton.setBounds(323, 84, 89, 23);
contentPane.add(btnNewButton);
```

Algoritmos de Búsqueda Backtracking

Ejemplo1: Resolver el Problema de las N Reinas

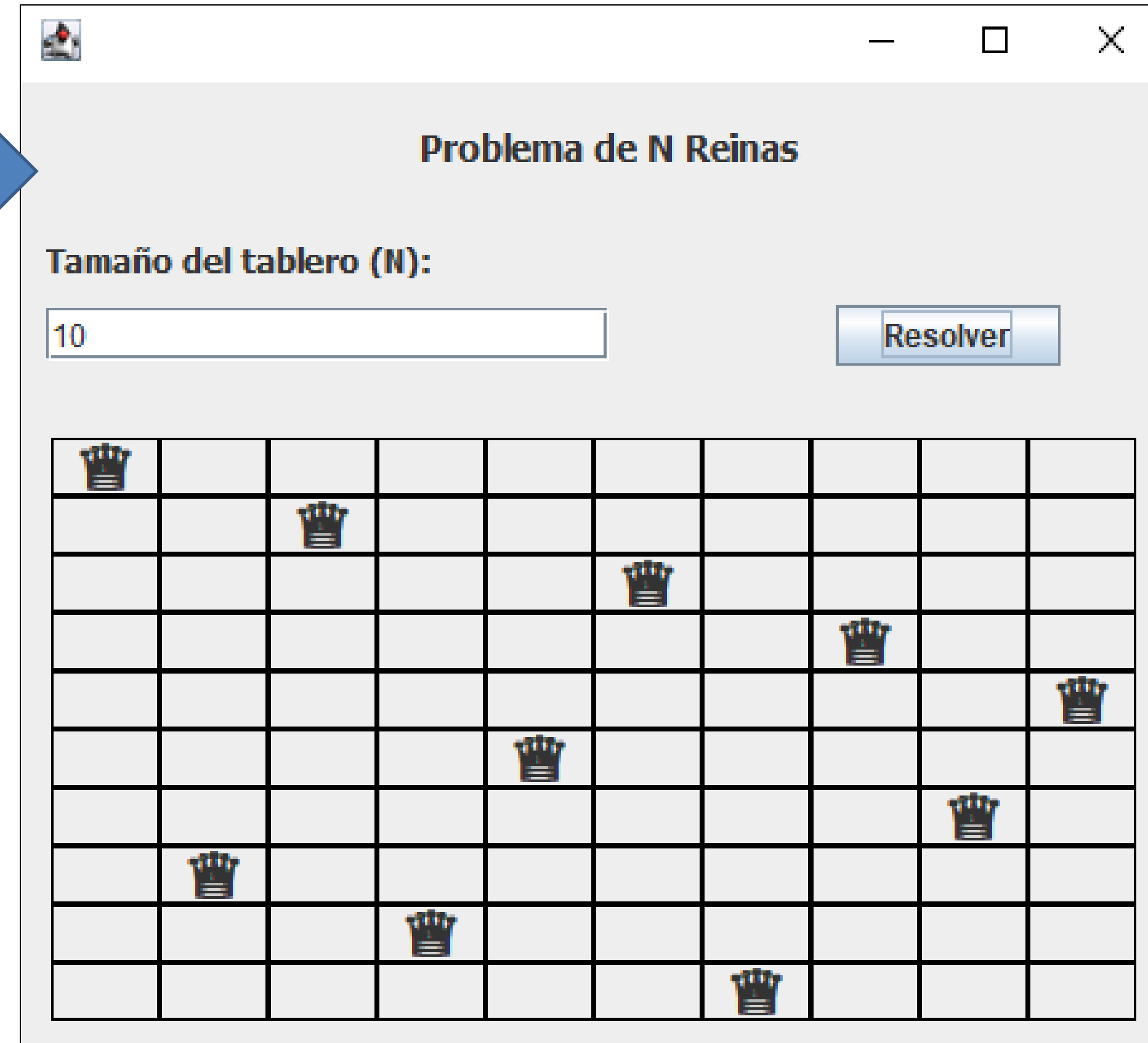


Problema de N Reinas

Tamaño del tablero (N):

Resolver

The image shows a software window titled "Problema de N Reinas". It contains a text input field for "Tamaño del tablero (N):" which is currently empty. To the right of the input field is a button labeled "Resolver". Below the input field is a large dashed rectangular box, intended for displaying the chessboard solution.



Problema de N Reinas

Tamaño del tablero (N):

Resolver

♔									
		♔							
					♔				
							♔		
									♔
				♔					
								♔	
	♔								
			♔						
						♔			

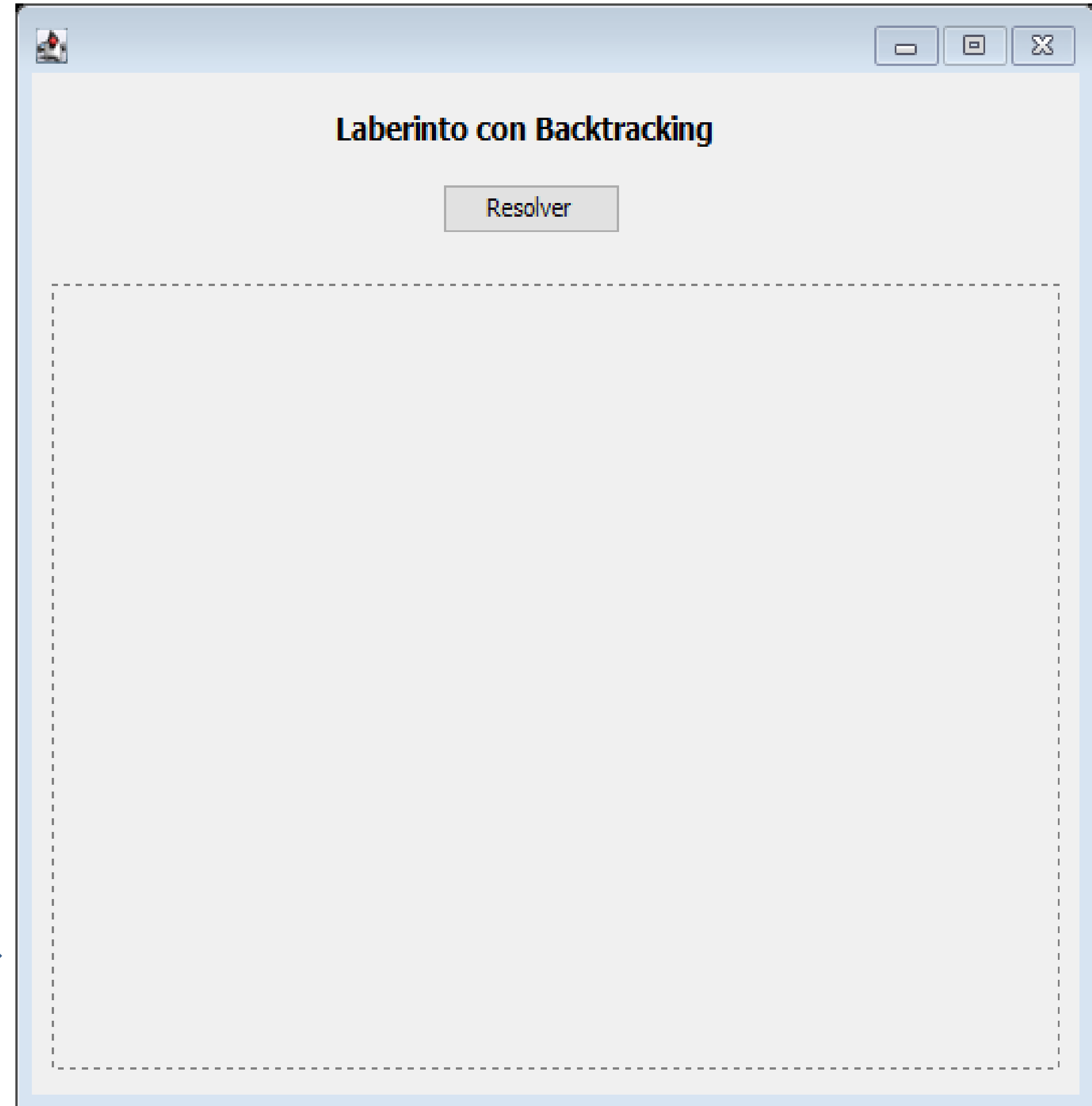
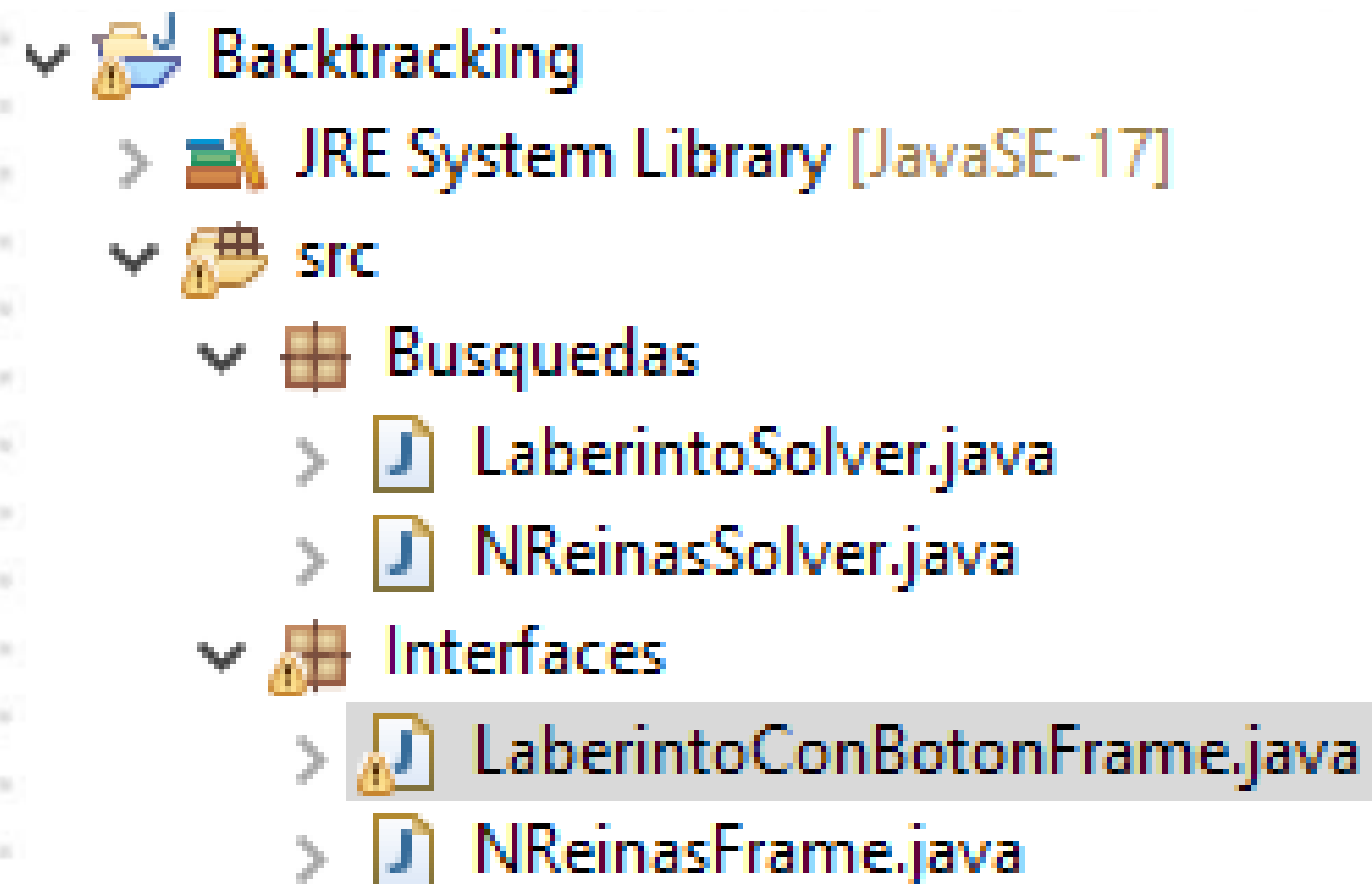
The image shows the same software window as the left one, but with the input field containing the value "10". The "Resolver" button is now highlighted in blue. Below the input field, a 10x10 chessboard is displayed, showing a valid solution to the 10-Queens problem. The queens are placed at the following positions (row, column): (1,1), (2,3), (3,6), (4,8), (5,10), (6,5), (7,9), (8,2), (9,4), and (10,7).

Algoritmos de Búsqueda Backtracking

Ejemplo2: Resolver un Laberinto (Backtracking visual)

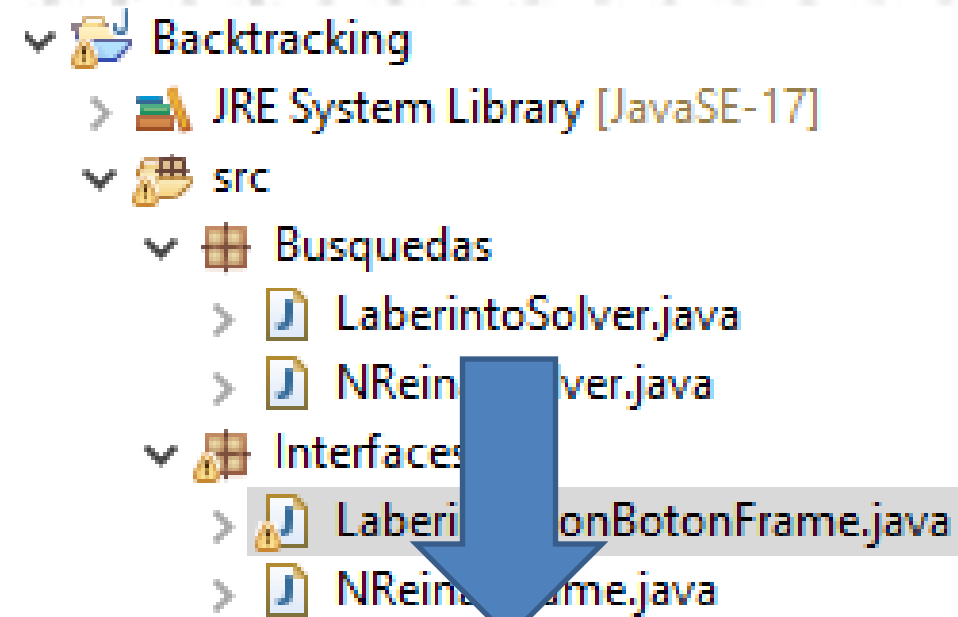
Descripción:

- El programa resuelve un laberinto definido como una matriz.
- Muestra el camino desde el inicio hasta el fin usando backtracking.



Algoritmos de Búsqueda Backtracking

Ejemplo2: Resolver un Laberinto (Backtracking visual)



```
1 package Busquedas;
2
3 public class LaberintoSolver {
4
5     private int[][] laberinto;
6     private int[][] solucion;
7     private int N;
8
9     public LaberintoSolver(int[][] laberinto) {
10         this.laberinto = laberinto;
11         this.N = laberinto.length;
12         this.solucion = new int[N][N];
13     }
14
15     public boolean resolver() {
16         return buscarCamino(0, 0);
17     }
18 }
```

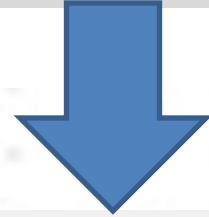
```
19 private boolean buscarCamino(int x, int y) {
20     if (x == N - 1 && y == N - 1 && laberinto[x][y] == 1) {
21         solucion[x][y] = 1;
22         return true;
23     }
24
25     if (esValido(x, y)) {
26         solucion[x][y] = 1;
27
28         if (buscarCamino(x + 1, y)) return true;
29         if (buscarCamino(x, y + 1)) return true;
30         if (buscarCamino(x - 1, y)) return true;
31         if (buscarCamino(x, y - 1)) return true;
32
33         solucion[x][y] = 0; // backtrack
34     }
35     return false;
36 }
```

```
38 private boolean esValido(int x, int y) {
39     return x >= 0 && y >= 0 && x < N && y < N && laberinto[x][y] == 1;
40 }
41
42 public int[][] getSolucion() {
43     return solucion;
44 }
45
46 }
```

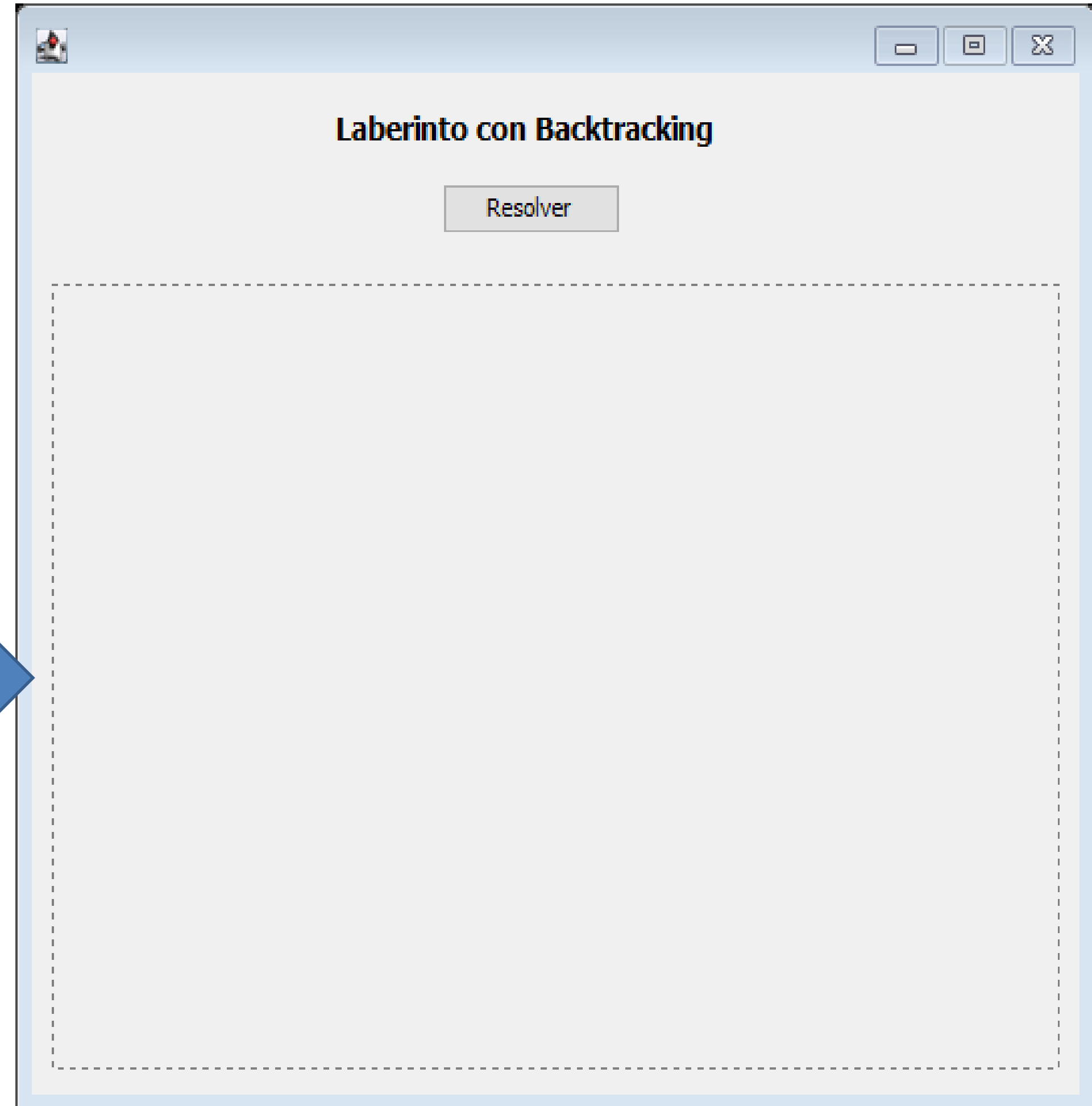
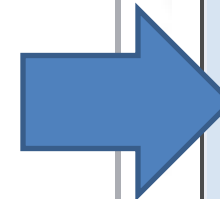
Algoritmos de Búsqueda Backtracking

Ejemplo2: Resolver un Laberinto (Backtracking visual)

- ▼ Backtracking
 - > JRE System Library [JavaSE-17]
 - ▼ src
 - ▼ Busquedas
 - > LaberintoSolver.java
 - > NReinasSolver.java
 - ▼ Interfaces
 - > LaberintoConBotonFrame.java
 - > NReinasFrame.java

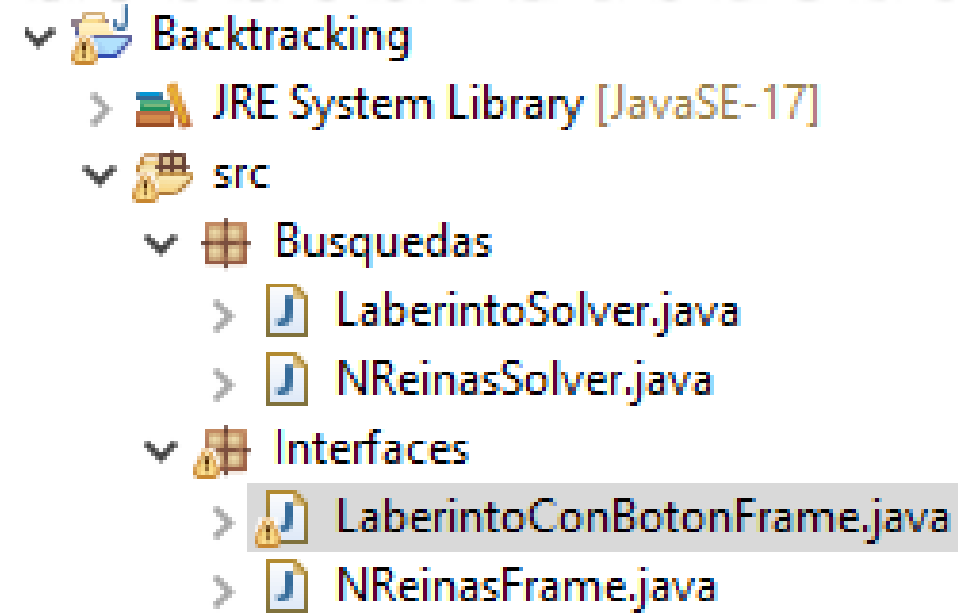


- Components
- ▼ (javax.swing.JFrame)
 - ▼ contentPane
 - panel
 - ? lblNewLabel - "Laberinto con Backtracking"
 - ⓧ btnResolver - "Resolver"



Algoritmos de Búsqueda Backtracking

Ejemplo2: Resolver un Laberinto (Backtracking visual)

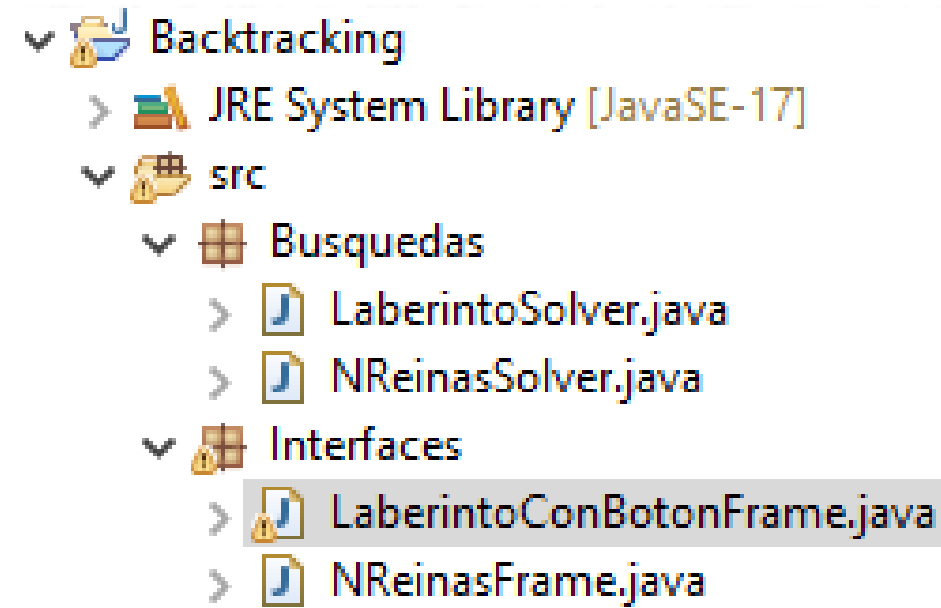


```
1 package Interfaces;
2
3 import Busquedas.LaberintoSolver;
4
5 import java.awt.BorderLayout;
6 import java.awt.Color;
7 import java.awt.EventQueue;
```

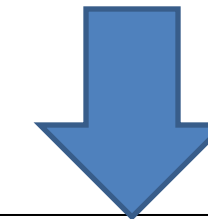
```
9 import javax.swing.JFrame;
10 import javax.swing.JPanel;
11 import javax.swing.SwingConstants;
12 import javax.swing.border.EmptyBorder;
13 import javax.swing.BorderFactory;
14 import javax.swing.JButton;
15 import javax.swing.JLabel;
16 import javax.swing.JOptionPane;
17
18 import java.awt.Font;
19 import java.awt.GridLayout;
20 import java.awt.event.ActionListener;
21 import java.awt.event.ActionEvent;
```

Algoritmos de Búsqueda Backtracking

Ejemplo2: Resolver un Laberinto (Backtracking visual)



```
	JButton btnResolver = new JButton("Resolver");
	btnResolver.addActionListener(new ActionListener() {
		public void actionPerformed(ActionEvent e) {
			resolverLaberinto();
		}
	});
```

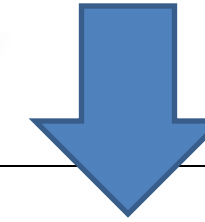


```
23 public class LaberintoConBotonFrame extends JFrame {
24
25     private static final long serialVersionUID = 1L;
26     private JPanel contentPane;
27
28     private int[][] laberinto = {
29         {1, 0, 1, 1},
30         {1, 1, 1, 0},
31         {0, 1, 0, 1},
32         {1, 1, 1, 1}
33     };
34     private int[][] solucion;
```

```
private void resolverLaberinto() {
    LaberintoSolver solver = new LaberintoSolver(laberinto);
    if (solver.resolver()) {
        solucion = solver.getSolucion();
        mostrarMatriz(solucion);
    } else {
        JOptionPane.showMessageDialog(null, "No hay solución posible.");
    }
}
```


Algoritmos de Búsqueda Backtracking

Ejemplo2: Resolver un Laberinto (Backtracking visual)

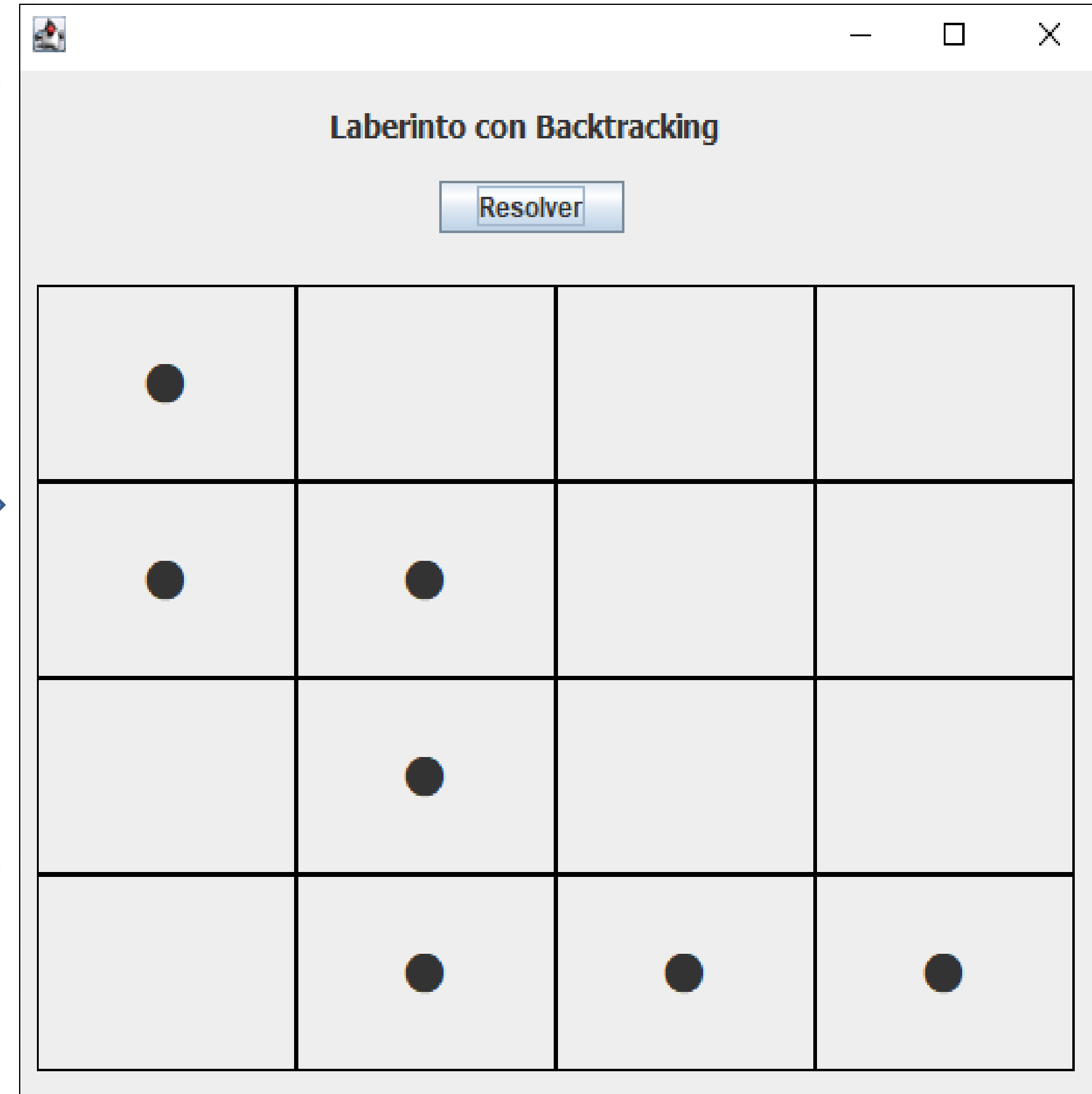
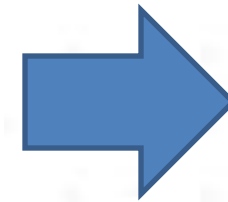
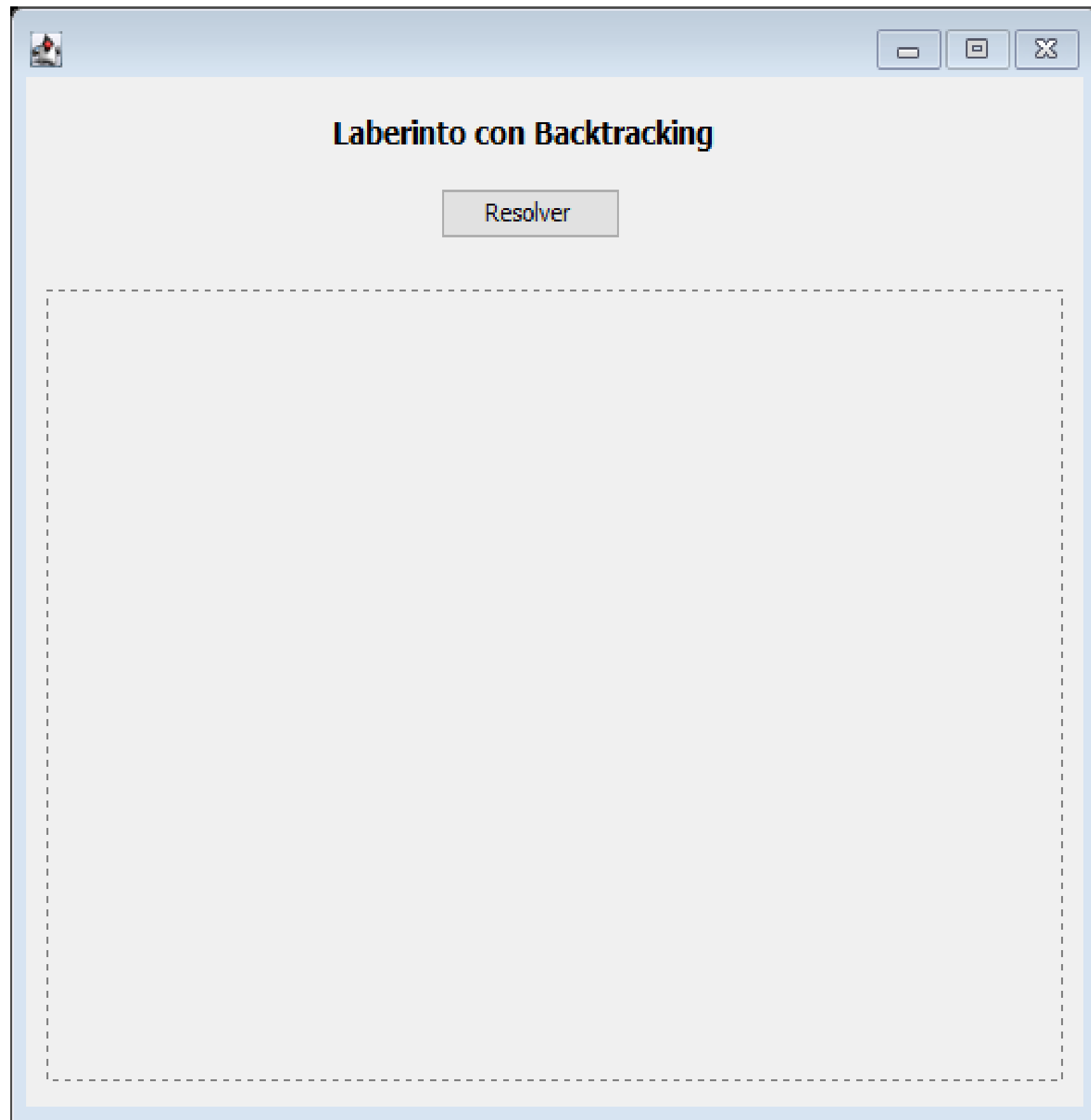


```
private void mostrarMatriz(int[][] matriz) {
    int N = matriz.length;
    panel.removeAll();
    panel.setLayout(new GridLayout(N, N));
    for (int[] fila : matriz) {
        for (int celda : fila) {
            JLabel label = new JLabel(celda == 1 ? "●" : "", SwingConstants.CENTER);
            label.setFont(new Font("SansSerif", Font.BOLD, 24));
            label.setBorder(BorderFactory.createLineBorder(Color.BLACK));
            panel.add(label);
        }
    }
    panel.revalidate();
    panel.repaint();
}

btnResolver.setBounds(204, 49, 89, 23);
contentPane.add(btnResolver);
```

Algoritmos de Búsqueda Backtracking

Ejemplo2: Resolver un Laberinto (Backtracking visual)





**Universidad
Tecnológica
del Perú**